Embedded Module Architecture 設計仕様書

ESP32-S3 CAM モジュールインターフェース設計パターン

バージョン	2.6
作成日	2026 年 3 月 15 日
著者	Shiozawa
ステータス	改訂中

本ドキュメントは、ESP32-S3 N16R8 CAM 開発ボードを題材に、
組み込みシステム向け Module インターフェースの
設計思想と API を定義する仕様書です。

改訂履歴

版	日付	著者	変更内容
1.0	2026-03-11	Shiozawa	初版作成。IModule、ModuleTimer、SystemData パターンの設計仕様を定義
1.1	2026-03-11	Shiozawa	IModule をテンプレート化（IModule¡T¿）。コア層の汎用性向上、参照渡しに変更
1.2	2026-03-11	Shiozawa	背景・目的セクション更新。共同開発の課題、部品化・規格統一の目的を追記
1.3	2026-03-12	Shiozawa	モジュール Config 構造体パターンを追加。ProjectConfig.h に Config 定義と SystemData を統合
1.4	2026-03-12	Shiozawa	ループ実行手順の 3 フェーズモデルを追加。入力→ロジック→出力の実行順序ガイドラインを整備
1.5	2026-03-12	Shiozawa	全モジュール再利用前提の設計方針を追加。概要・目的セクションを更新
1.6	2026-03-13	Shiozawa	デザインスタイルをリファレンステンプレート準拠に刷新。色定義・見出し・ヘッダー/フッター・tcolorbox スタイルを統一
1.7	2026-03-13	Shiozawa	既知の制約に対する設計上の対応方針を策定。enabled フラグ追加、データアクセスルール定義、割り込み実行パターン・内部バッファパターン・エラー復帰パターンを追加
1.8	2026-03-13	Shiozawa	設計ルールをアーキテクチャセクションに集約、制約セクションをリファクタリング
1.9	2026-03-13	Shiozawa	通信バスパターンを変更：バスポインタを Config 構造体からコンストラクタ引数に移行
2.0	2026-03-13	Shiozawa	設計ルール拡充：deinit() 追加、Data 構造体デフォルト値、外部データアクセスパターン（volatile フラグ方式）、init 復帰パターン、ログ出力フォーマット、Config 変更タイミング制約、メモリ管理指針を追加
2.1	2026-03-13	Shiozawa	実装検証によるフィードバック反映：モジュールの.h/.cpp 分離ルール追加（循環依存対策）、PlatformIO 設定に-I include 追加、コード例を実装準拠に更新
2.2	2026-03-13	Shiozawa	SystemData.h を ProjectConfig.h から分離。モジュールの.cpp は SystemData.h のみ参照する設計に変更。仕様書の全コード例・ファイル構成を更新
2.3	2026-03-13	Shiozawa	IModule からテンプレートを除去。前方宣言ベースのシンプルなインターフェースに変更
2.4	2026-03-14	Shiozawa	TftModule（SPI TFT+ タッチ）、Mpu6500Module（IMU/I2C）、CameraModule（OV5640）を実装・追記
2.5	2026-03-15	Shiozawa	ServoModule、DriveMotorModule、ChassisModule（4 輪統合足回り制御）を追加
2.6	2026-03-15	Shiozawa	仕様書をアーキテクチャ仕様に特化。実装モジュール個別仕様を分離し、ファイル構成をパターン表記に変更

目次

改訂履歴 1

1 概要 4
- 1.1 プロジェクト名 . . . 4
- 1.2 コンセプト . . . 4
- 1.3 設計思想 . . . 4

2 背景・課題 5
- 2.1 組み込み開発における一般的な課題 . . . 5
- 2.2 共同開発における課題 . . . 5
- 2.3 ESP32-S3 CAM の特徴 . . . 6

3 目的 6
- 3.1 主目的 . . . 6
- 3.2 副次的目的 . . . 7
- 3.3 範囲 . . . 7

4 システムアーキテクチャ 7
- 4.1 全体構造 . . . 7
- 4.2 実行フロー . . . 7
- 4.3 設計パターン . . . 9

5 ファイル構成 19
- 5.1 標準ディレクトリ構造 . . . 20
- 5.2 ファイル役割一覧 . . . 20
- 5.3 ライブラリ構成方針 . . . 20

6 技術スタック 21
- 6.1 PlatformIO 設定 . . . 21

7 API 仕様 21
- 7.1 IModule インターフェース . . . 22
- 7.2 ModuleTimer クラス . . . 24
- 7.3 ProjectConfig ファイルと SystemData . . . 26
- 7.4 モジュール Config 構造体 . . . 27

8 実行モデル詳細 29
- 8.1 ライフサイクル . . . 29
- 8.2 ループ実行手順（3 フェーズモデル） . . . 30
- 8.3 タイミング制御 . . . 31
- 8.4 モジュール間データフロー . . . 32
- 8.5 割り込み実行パターン . . . 32
- 8.6 外部データアクセスパターン（スレッドセーフティ） . . . 33
- 8.7 内部バッファパターン . . . 35

8.8 モジュール実装テンプレート 36

9 **既知の制約** 37

9.1 SystemData の肥大化 37
9.2 モジュール間の依存関係 37
9.3 リアルタイム性の限界 37
9.4 シングルスレッド制約 37
9.5 ロジックフェーズのスケーラビリティ 37
9.6 動的なモジュール管理 37
9.7 エラー復帰パターン 38
9.8 Config の実行時変更 38
9.9 テスタビリティ 38
9.10 メモリ管理 38

1 概要

1.1 プロジェクト名

Embedded Module Architecture — ESP32-S3 CAM サンプルプロジェクト

1.2 コンセプト

本プロジェクトは、組み込みシステムにおける **Module インターフェース設計パターン**のリファレンス実装である。ESP32-S3 N16R8 CAM 開発ボードを題材に、カメラ・センサー・LED・LCD 等の多様なハードウェアモジュールを統一的なインターフェースで管理する手法を示す。

1.3 設計思想

◆ 基本理念：どんなプロジェクトにも流用できる汎用的なコアクラス

IModule・ModuleTimer は**プロジェクトに一切依存しない**汎用コンポーネントとして設計する。`SystemData` は前方宣言のみで参照するため、プロジェクト固有のヘッダーに依存しない。コアクラスをコピーするだけで、どんな組み込みプロジェクトにもそのまま流用可能である。

◆ 基本理念：各モジュールはそれ単体で動作する道具

モジュールは他のモジュールの存在を知らず、**自身の SystemData サブ構造体のみ**を参照する。入力モジュールは「ハードウェアから値を読んで SystemData に書く」だけ、出力モジュールは「SystemData の値に従ってハードウェアを制御する」だけの単機能な道具である。「何をどう制御するか」というロジックはモジュールの外（ロジックフェーズ）で行う。

◆ 基本理念：すべてのモジュールは再利用可能な部品

コアクラス（IModule・ModuleTimer）だけでなく、**IModule を継承したすべての子クラス（各モジュール実装）も、別のプロジェクト・別の開発者にそのまま流用できる**ことを前提として設計する。一度作成したモジュールはプロジェクト間で共有し、誰でも同じように使える「共有部品」として扱う。

この前提を守るためのルール：

- モジュール内にプロジェクト固有のロジックを記述しない（入力は読み取るだけ、出力は SystemData に従うだけ）
- ハードウェア依存の設定（ピンアサイン等）は Config 構造体で外部から注入する
- モジュールが参照する SystemData サブ構造体は、モジュール自身のヘッダ内で定義し、モジュールと一緒に持ち運べるようにする
- モジュールが他のモジュールのサブ構造体を直接参照しない

本アーキテクチャの根幹となる 5 つの要素：

1. **IModule インターフェース** — `SystemData` の前方宣言のみを使用し、プロジェクト固有の型に依存せず、全モジュールが `init()` と `update(SystemData& data)` を実装する統一的なライフサイクル管理
2. **ModuleTimer クラス** — `delay()` を一切使用しないノンブロッキングなタイマー制御
3. **SystemData パターン** — プロジェクト固有の共有状態をサブ構造体で一元管理し、モジュール間のデータ連携を実現

4. **モジュール Config パターン** — 各モジュールのピンアサインや設定を Config 構造体として定義し、コンストラクタで受け取ることでハードウェア構成を明示的かつ一元的に管理
5. **3 フェーズ実行モデル** — ループ内の処理を「入力→ロジック→出力」の 3 段階に分離し、モジュールの独立性を保証する実行手順
6. **通信バスパターン** — SPI・I2C 等の共有バスインスタンスを `main.cpp` で一元管理し、ポインタでモジュールに渡すことでバスの競合を防止

この設計により、以下を実現する：

- **コアクラスの完全な汎用性**（プロジェクト非依存）
- **全モジュールのプロジェクト横断的な再利用性**（子クラスも含め、作ったモジュールをそのまま別プロジェクトで共有可能）
- **モジュールの独立性**（各モジュールは単体で動作する道具）
- 入出力両方を持つデバイスもモジュール分離により独立性を維持
- モジュールの追加・削除が容易
- 処理順序が決定論的（入力→ロジック→出力の 3 フェーズ）
- 複数回出力や処理の重複が発生しない
- ゲームループ（ECS）に類似した、組み込みに適した実行モデル
- 参照渡し（`&`）による null ポインタ問題の排除
- **ピンアサインの一元管理**（Config 構造体により回路設計者がピン定義ファイルだけで把握可能）
- **通信バスの安全な共有**（SPI・I2C バスを `main.cpp` で一元管理し、ポインタで渡すことで競合を防止）

2 背景・課題

2.1 組み込み開発における一般的な課題

組み込みシステムの開発において、以下のような問題が頻繁に発生する：

1. **delay() の乱用**
   ブロッキング関数である `delay()` を使用すると、その間は他の処理が完全に停止する。複数のセンサーやアクチュエータを同時に制御する場合、`delay()` は致命的なボトルネックとなる。
2. **モジュール間の密結合**
   各ハードウェアモジュールの制御コードが `loop()` 内に直接記述されると、コードの肥大化・複雑化が進み、モジュールの追加・削除・変更が困難になる。
3. **出力の重複・不整合**
   統一的なライフサイクル管理がない場合、同一フレーム内で複数回のセンサー読み取りや出力が発生するなど、処理の重複や不整合が起こりやすい。
4. **グローバル変数の氾濫**
   モジュール間のデータ共有にグローバル変数を多用すると、依存関係が不明瞭になり、バグの原因となる。

2.2 共同開発における課題

授業やチーム開発において、複数人で組み込みプロジェクトを進める際に以下の問題が顕在化する：

1. **実装スタイルの不統一**
   「A さんは関数のみで実装」「B さんはクラスで実装」「C さんは AI に生成させた独自構造」のように、メン

バーごとに実装スタイルがバラバラになると、統合時に重大な問題が発生する。インターフェースの不一致、命名規則の違い、データの受け渡し方法の齟齬など、結合の段階で大幅な手戻りが生じる。

2. **AI 活用時のコード品質のばらつき**
   近年、AI コード生成ツールの普及により、プログラミング初心者でもコードを生成できるようになった。しかし、各メンバーがそれぞれ自由に AI に指示を出すと、生成されるコードの構造・スタイルが統一されず、プロジェクト全体の一貫性が失われる。
3. **機能分担の曖昧さ**
   明確なインターフェース規約がないと、「誰がどの機能を担当し、どのような API で提供するか」が不明確になる。結果として、機能の重複や抜け漏れが発生しやすくなる。

2.3 ESP32-S3 CAM の特徴

本プロジェクトで使用する ESP32-S3 N16R8 CAM 開発ボードは、以下の特徴を持つ：

項目	仕様
MCU	ESP32-S3（デュアルコア Xtensa LX7）
Flash	16MB（N16）
PSRAM	8MB（R8、OPI 接続）
通信	WiFi + Bluetooth 5.0
カメラ	OV2640 / OV5640 対応
フレームワーク	Arduino

表 1: ESP32-S3 N16R8 CAM 主要スペック

多機能なハードウェアを効率的に管理するために、統一的なモジュール管理アーキテクチャが求められる。

3 目的

本プロジェクトの目的は以下の通りである：

3.1 主目的

1. **プログラムの部品化による汎用性の向上**
   IModule インターフェースと ModuleTimer クラスを、**どんなプロジェクトにもそのまま流用できる**汎用的なコアコンポーネントとして設計する。プロジェクト固有の型に一切依存せず、コアクラスをコピーするだけで別のマイコンプラットフォーム・別のプロジェクトでも即座に利用可能な部品化を実現する。

   さらに、コアクラスだけでなく、**IModule を継承したすべての子クラス（各モジュール実装）も再利用を前提**として設計する。一度作成したモジュールは、別のプロジェクト・別の開発者にそのまま共有・流用できる「共有部品」として扱う。これにより、モジュールのエコシステムが蓄積され、新しいプロジェクトの立ち上げが加速する。
2. **共同開発における規格の確立**
   複数人での組み込み開発において、メンバー全員が同じインターフェース規約に従うことで、実装スタイルの統一を実現する。「このモジュールを作って」と依頼する際に、`IModule` を継承した `init()` と `update()` を実装するだけという明確なルールがあれば、初心者でも AI に適切な指示を出しやすく、かつ、モジュール単

位で機能を分担しやすくなる。

◆ 共同開発でのメリット

- 「○○さんは LED モジュール」「△△さんは温度センサーモジュール」と明確に分担できる
- 全員が同じインターフェースに従うため、統合時のコンフリクトが最小限
- AI にコード生成を依頼する場合も、IModule の規約を提示すれば一貫した出力が得られる
- モジュール単位でテスト・交換が可能なため、個人の作業が他に影響しにくい

3.2 副次的目的

1. **教育・学習用のサンプル**
   組み込み開発の初学者が、モジュール化・非ブロッキング処理・データフロー管理を学べる具体的なコード例を提供する。
2. **ESP32-S3 CAM の活用例**
   カメラ、センサー、LED、LCD 等の周辺機器を組み合わせた実用的なサンプルアプリケーションを構築する。

3.3 範囲

本プロジェクトが提供するもの：

- IModule インターフェースの定義と仕様
- ModuleTimer クラスの実装
- SystemData パターンの設計指針
- ESP32-S3 CAM を用いたサンプルモジュール群

4 システムアーキテクチャ

4.1 全体構造

本アーキテクチャは、以下のコア要素で構成される：

1. **IModule**（インターフェース） — モジュールのライフサイクルを定義。プロジェクト非依存
2. **ModuleTimer**（ユーティリティクラス） — ノンブロッキングなタイマー制御。プロジェクト非依存
3. **SystemData**（共有データ構造体） — モジュール間のデータ連携。プロジェクト固有
4. **モジュール Config**（設定構造体） — 各モジュールのピンアサインや設定を構造体で定義。プロジェクト固有

▷ コア層とプロジェクト層の分離

`IModule` と `ModuleTimer` は**コア層**に属し、プロジェクト固有の型を一切知らない。
`SystemData` とモジュール Config は**プロジェクト層**に属し、前者は使用するモジュールのサブ構造体を集約し、後者はハードウェア構成（ピンアサイン等）を定義する。
`IModule` は `SystemData` を前方宣言で参照するため、コア層の完全な独立性を実現している。

4.2 実行フロー

プログラムの実行フローは、ゲームループパターンに基づき、3 フェーズモデルで構成される：

```cpp
// main.cpp
#include "IModule.h"
#include "ProjectConfig.h"  // SystemData + Config定義
#include "TempSensorModule.h"
#include "SwitchModule.h"
#include "LedModule.h"
#include "MotorModule.h"

// システムデータ
SystemData systemData;

// 入力モジュール（Config構造体を渡して生成）
TempSensorModule tempModule(TEMP_CONFIG);
SwitchModule swModule(SW_CONFIG);

// 出力モジュール
LedModule ledModule(LED_CONFIG);
MotorModule motorModule(MOTOR_CONFIG);

// 入力モジュール配列
IModule* inputModules[] = {
    &tempModule,
    &swModule,
};
const int INPUT_COUNT = sizeof(inputModules) / sizeof(inputModules[0]);

// 出力モジュール配列
IModule* outputModules[] = {
    &ledModule,
    &motorModule,
};
const int OUTPUT_COUNT = sizeof(outputModules) / sizeof(outputModules[0]);

// ロジックフェーズ（標準は関数として実装）
void applyPattern(SystemData& data) {
    // 入力SystemDataを読み、出力SystemDataを書き換える
    data.led.state = data.sw.isPressed;
    data.motor.speed = (data.temp.temperature > 30.0) ? 255 : 0;
}

void setup() {
    Serial.begin(115200);

    // 全入力モジュールを初期化（失敗時はリトライ後に無効化）
    const int MAX_RETRY = 3;
    for (int i = 0; i < INPUT_COUNT; i++) {
        bool success = false;
        for (int r = 0; r < MAX_RETRY; r++) {
            if (inputModules[i]->init()) { success = true; break; }
            delay(100);
        }
        if (!success) {
            Serial.printf("Input Module %d: init failed, disabled\n", i);
            inputModules[i]->enabled = false;
```

```
        }
    }
    // 全出力モジュールを初期化
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        bool success = false;
        for (int r = 0; r < MAX_RETRY; r++) {
            if (outputModules[i]->init()) { success = true; break; }
            delay(100);
        }
        if (!success) {
            Serial.printf("Output Module %d: init failed, disabled\n", i);
            outputModules[i]->enabled = false;
        }
    }
}

void loop() {
    // 1. 入力フェーズ：センサー値をSystemDataに書き込む
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {
            inputModules[i]->update(systemData);
        }
    }
    // 2. ロジックフェーズ：入力→出力のSystemData変換
    applyPattern(systemData);
    // 3. 出力フェーズ：SystemDataの値に従いハードウェア制御
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (outputModules[i]->enabled) {
            outputModules[i]->update(systemData);
        }
    }
}
```

Listing 1: 3 フェーズモデルによる実行フロー

4.3 設計パターン

4.3.1 モジュール配列パターン

`inputModules[]` と `outputModules[]` の 2 つの配列でモジュールを管理する。配列の順序がそのまま各フェーズ内の実行順序となる。これにより：

- 入力と出力の処理順序が明示的かつ決定論的
- 入力→ロジック→出力のデータの流れが保証される
- 1 ループにつき各モジュール 1 回の更新が保証される
- 各モジュールが独立した「道具」として機能する

▷ 重要：処理順序

各フェーズ内での配列順序はデータフローの順序を意味する。
例：温度センサーの値を補正センサーで校正する場合、温度センサーを補正センサーより前に配置する。
ただし、入力と出力のクロス参照（出力モジュールが入力の SystemData を読む等）は行わない。

4.3.2 モジュールのファイル構成

各モジュールは**ヘッダー（.h）と実装（.cpp）に分離**する。これは SystemData の循環依存を回避するための必須ルールである。

！ 循環依存の問題

モジュールクラスは IModule を継承するため SystemData の型が必要となる。しかし SystemData は各モジュールの Data 構造体を含むため、モジュールヘッダーをインクルードする。
ヘッダー内にメソッドをインライン実装すると、LedModule.h → SystemData → LedModule.h の循環が発生してコンパイルエラーとなる。

SystemData を include/SystemData.h に分離し、モジュールの.cpp では SystemData.h のみをインクルードすることで循環依存を回避する。ProjectConfig.h は Config インスタンスの定義のみを担当し、main.cpp だけが参照する。

◆ モジュールファイル構成のルール

1. **ヘッダー（.h）**：Config 構造体、Data 構造体、SystemData の前方宣言、クラス宣言（メソッドはプロトタイプのみ）
2. **実装（.cpp）**：SystemData.h をインクルードし、メソッドを実装（ProjectConfig.h はインクルードしない）
3. ヘッダーでは<Arduino.h>と"IModule.h"をインクルードする
4. ヘッダーでは SystemData.h をインクルードしない（循環依存を防ぐため）

```cpp
// --- LedModule.h（ヘッダー：宣言のみ） ---
#pragma once
#include <Arduino.h>
#include "IModule.h"

struct LedConfig {
    uint8_t ledPin;
};

struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

struct SystemData;  // 前方宣言（実体はSystemData.hで定義）

class LedModule : public IModule {
private:
    LedConfig _config;
public:
    LedModule(const LedConfig& config);
    bool init() override;
    void update(SystemData& data) override;
};

// --- LedModule.cpp（実装：SystemData.hで完全な定義を取得） ---
#include "LedModule.h"
#include "SystemData.h"
```

```

LedModule::LedModule(const LedConfig& config) : _config(config) {}

bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);
    Serial.println("[Led] init OK");
    return true;
}

void LedModule::update(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state ? HIGH : LOW);
}
```

Listing 2: モジュールのファイル構成例

4.3.3 SystemData パターン

各モジュールは固有のサブ構造体を定義し、それらをプロジェクト固有の SystemData 構造体に集約する。

```
// 各モジュールが自身のデータ構造体を定義（ヘッダー内）
// メンバには必ずデフォルト値を明示する
// LedModule.h 内
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

// TempSensorModule.h 内
struct TempData {
    float temperature = -999.0f;  // 無効値
    bool isValid = false;         // まだ読み取っていない
    bool hasError = false;
};

// SystemData.h 内: 各モジュールのData構造体を集約
struct SystemData {
    LedData led;
    TempData temp;
    // 使用するモジュールの構造体を追加するだけ
};
```

Listing 3: SystemData パターンの構造

4.3.4 モジュール Config パターン

各モジュールは、自身に必要なピンアサインや設定を Config 構造体として定義する。モジュールのコンストラクタではピン番号を個別に受け取るのではなく、Config 構造体を受け取ることで、ハードウェア構成を明示的かつ安全に管理する。

◆ Config パターンの利点

- コンストラクタの引数順序を覚える必要がない（構造体のメンバ名で明示的に指定）
- 全モジュールのピンアサインが ProjectConfig.h に集約され、一覧性が高い

- 回路設計担当者は ProjectConfig.h の Config 定義セクションだけを見れば回路設計が可能
- 将来的にピン以外の設定（I2C アドレス、動作モード等）も Config 構造体に追加して拡張可能

```
// LedModule.h 内 — Config構造体とクラス宣言
struct LedConfig {
    uint8_t ledPin;       // LED出力ピン
};

struct SystemData;  // 前方宣言

class LedModule : public IModule {
private:
    LedConfig _config;
public:
    LedModule(const LedConfig& config);  // 実装は.cppで
    bool init() override;
    void update(SystemData& data) override;
};

// ProjectConfig.h 内 — プロジェクトのピンアサインを集約
const LedConfig LED_CONFIG = { .ledPin = 13 };

// main.cpp — Config定数を渡してインスタンス生成
LedModule ledModule(LED_CONFIG);
```

Listing 4: モジュール Config パターンの例

4.3.5 通信バスパターン

SPI・I2C 等の通信バスは複数モジュールで共有されるリソースであり、個々のモジュールが所有・初期化すべきではない。バスインスタンスを main.cpp で生成し、**コンストラクタ引数**でモジュールに渡す。

◆ 通信バスを共有リソースとして管理する理由

- 複数モジュールが同じバスを使う場合、各モジュールが begin() すると競合・二重初期化が発生する
- バスのピンアサインやインスタンスがモジュール内にハードコードされると、モジュールの再利用性が損なわれる
- バスの生成と初期化を main.cpp に集約することで、共有リソースの管理が明確になる

◆ バスポインタを Config 構造体に含めない理由

Config 構造体のインスタンスは ProjectConfig.h で定義する。もし Config 構造体にバスポインタを含めると、ProjectConfig.h より前にバスインスタンスをグローバルスコープで生成し extern 宣言する必要があり、インクルード順序の制約が生じる。
バスの紐付けをコンストラクタ引数に分離することで、Config 構造体はピンアサイン等の純粋な設定値のみを保持し、ProjectConfig.h はバスインスタンスの存在を知る必要がなくなる。

SPI バスの例

```
// SpiSensorModule.h
#pragma once
#include <SPI.h>
#include "IModule.h"
```

```cpp
#include "ModuleTimer.h"

// --- Config構造体（デバイス固有設定のみ、バスポインタは含めない） ---
struct SpiSensorConfig {
    uint8_t csPin;            // チップセレクトピン
    uint32_t clockSpeed;      // SPIクロック速度(Hz)
    unsigned long interval;   // 読み取り間隔(ms)
};

// --- Data構造体 ---
struct SpiSensorData {
    uint16_t rawValue = 0;
    bool hasError = false;
};

class SpiSensorModule : public IModule {
private:
    SpiSensorConfig _config;
    SPIClass* _spi;           // コンストラクタで受け取るバスポインタ
    ModuleTimer _readTimer;
public:
    // Config + バスポインタをコンストラクタで受け取る
    SpiSensorModule(const SpiSensorConfig& config, SPIClass* spi)
        : _config(config), _spi(spi) {}

    bool init() override {
        // CSピンの設定のみ行う（bus.begin()は呼ばない）
        pinMode(_config.csPin, OUTPUT);
        digitalWrite(_config.csPin, HIGH);
        _readTimer.setTime();
        return true;
    }

    void update(SystemData& data) override {
        if (_readTimer.getNowTime() >= _config.interval) {
            _readTimer.setTime();
            // beginTransaction/endTransactionで排他制御
            _spi->beginTransaction(
                SPISettings(_config.clockSpeed, MSBFIRST, SPI_MODE0)
            );
            digitalWrite(_config.csPin, LOW);
            data.spiSensor.rawValue = _spi->transfer16(0x00);
            digitalWrite(_config.csPin, HIGH);
            _spi->endTransaction();
        }
    }
};
```

Listing 5: SPI バスを使用するモジュールの実装例

I2C バスの例

```cpp
// I2cSensorModule.h
#pragma once
#include <Wire.h>
```

```cpp
#include "IModule.h"
#include "ModuleTimer.h"

// --- Config構造体（デバイス固有設定のみ、バスポインタは含めない） ---
struct I2cSensorConfig {
    uint8_t address;          // I2Cデバイスアドレス
    unsigned long interval;   // 読み取り間隔(ms)
};

// --- Data構造体 ---
struct I2cSensorData {
    float value = 0.0f;
    bool hasError = false;
};

class I2cSensorModule : public IModule {
private:
    I2cSensorConfig _config;
    TwoWire* _wire;           // コンストラクタで受け取るバスポインタ
    ModuleTimer _readTimer;
public:
    // Config + バスポインタをコンストラクタで受け取る
    I2cSensorModule(const I2cSensorConfig& config, TwoWire* wire)
        : _config(config), _wire(wire) {}

    bool init() override {
        // バスの初期化は行わない（main.cppで実施済み）
        _readTimer.setTime();
        return true;
    }

    void update(SystemData& data) override {
        if (_readTimer.getNowTime() >= _config.interval) {
            _readTimer.setTime();
            _wire->beginTransmission(_config.address);
            _wire->write(0x00);  // レジスタアドレス
            if (_wire->endTransmission() != 0) {
                data.i2cSensor.hasError = true;
                return;
            }
            _wire->requestFrom(_config.address, (uint8_t)2);
            if (_wire->available() >= 2) {
                uint8_t high = _wire->read();
                uint8_t low = _wire->read();
                data.i2cSensor.value =
                    (float)((high << 8) | low) / 100.0;
                data.i2cSensor.hasError = false;
            }
        }
    }
};
```

Listing 6: I2C バスを使用するモジュールの実装例

main.cpp でのバス生成・初期化

```cpp
// main.cpp
#include <SPI.h>
#include <Wire.h>
#include "ProjectConfig.h"
#include "SpiSensorModule.h"
#include "I2cSensorModule.h"

// バスインスタンスの生成（グローバルスコープ）
SPIClass spi(HSPI);
TwoWire wire(0);

// モジュール生成（Config + バスポインタをコンストラクタで渡す）
SpiSensorModule spiModule(SPI_SENSOR_CONFIG, &spi);
I2cSensorModule i2cModule(I2C_SENSOR_CONFIG, &wire);

void setup() {
    Serial.begin(115200);

    // バス初期化（全モジュールの init() より前）
    spi.begin(SCK_PIN, MISO_PIN, MOSI_PIN);
    wire.begin(SDA_PIN, SCL_PIN);

    // モジュール初期化
    spiModule.init();
    i2cModule.init();
}
```

Listing 7: main.cpp での通信バス生成と初期化

▷ 通信バスパターンのルール

1. バスインスタンスは main.cpp のグローバルスコープで生成する
2. bus.begin() は setup() 内で、全モジュールの init() より前に呼ぶ
3. バスポインタは Config 構造体に含めず、**コンストラクタの引数**でモジュールに渡す
4. モジュールはバスポインタをメンバ変数（_spi, _wire 等）として保持する
5. モジュールの init() で bus.begin() を呼ばない（CS ピン設定等のみ行う）
6. SPI 使用時は beginTransaction()/endTransaction() で排他制御する

アンチパターン：モジュール内でバスを生成・初期化する

```cpp
// NG: モジュール内でバスインスタンスを生成・初期化している
class BadSpiModule : public IModule {
private:
    SPIClass _spi(HSPI);  // モジュールがバスを所有
public:
    bool init() override {
        _spi.begin(18, 19, 23);  // ピンがハードコード
        return true;
    }
    // 別のモジュールも同じバスを begin() すると競合する
};
```

Listing 8: アンチパターン：モジュール内でバスを生成（禁止）

アンチパターン：Config 構造体にバスポインタを含める

```
// NG: Config構造体にバスポインタを含めている
struct BadSpiConfig {
    SPIClass* spi;        // Configに含めてはいけない
    uint8_t csPin;
};

// ProjectConfig.hでextern宣言が必要になってしまう
extern SPIClass spi;
const BadSpiConfig BAD_CONFIG = { .spi = &spi, .csPin = 5 };
```

Listing 9: アンチパターン：Config にバスポインタ（禁止）

4.3.6 Data 構造体のデフォルト値ルール

各 {Module}Data 構造体のメンバには、必ずデフォルト値を明示する。これにより、update() が 1 度も呼ばれていない状態でも安全にデータを参照できる。

◆ デフォルト値の指針

- bool 型のフラグ: false で初期化（isValid = false, hasError = false）
- 数値型: 正常値と区別できる無効値で初期化（例: temperature = -999.0f）
- ポインタ: nullptr で初期化

4.3.7 データアクセスルール

update() 内では、**自モジュールのサブ構造体のみ**にアクセスする。他モジュールのサブ構造体への読み書きは**禁止**とする。モジュール間のデータ連携が必要な場合は、必ず**ロジックフェーズ**を経由する。

```
// OK: 自モジュールのサブ構造体のみアクセス
void TempSensorModule::update(SystemData& data) {
    data.temp.temperature = analogRead(_config.tempPin);
    data.temp.isValid = true;
}

// NG: 他モジュールのサブ構造体にアクセスしている
void TempSensorModule::update(SystemData& data) {
    data.temp.temperature = analogRead(_config.tempPin);
    data.led.state = true;  // 禁止：LEDはロジックフェーズで制御する
}
```

Listing 10: データアクセスルールの例

4.3.8 モジュール統合の原則

モジュール間の依存関係は存在しないものとして設計する。各モジュールは他のモジュールの存在を知らず、独立して動作する。依存関係がある場合は、それらを **1 つの大きなモジュールに統合**する。

例：カメラモジュールが SD カードを必要とする場合、「カメラ＋ SD カードモジュール」として 1 つにまとめる。

4.3.9 enabled フラグによる動的管理

IModule の enabled フラグにより、実行時のモジュール有効・無効切り替えを実現する。

- enabled = true（デフォルト）: 通常通り update() が呼ばれる

- enabled = false: ループ内で update() がスキップされる
- BLE 等の外部通信からモジュールの enabled を切り替えることで、動的な制御が可能

```
void loop() {
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {
            inputModules[i]->update(systemData);
        }
    }
    applyPattern(systemData);
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (outputModules[i]->enabled) {
            outputModules[i]->update(systemData);
        }
    }
}
```

Listing 11: enabled フラグの使用例

4.3.10 エラーハンドリングパターン

init() 失敗時 規定回数（例：3 回）までリトライし、上限を超えた場合は enabled = false に設定してモジュールを無効化する。シリアルモニタにエラーログを出力する。

```
// setup() 内
const int MAX_RETRY = 3;
for (int i = 0; i < INPUT_COUNT; i++) {
    bool success = false;
    for (int retry = 0; retry < MAX_RETRY; retry++) {
        if (inputModules[i]->init()) {
            success = true;
            break;
        }
        delay(100);  // init()のリトライ間隔（setup内のみdelay許容）
    }
    if (!success) {
        Serial.printf("Input Module %d: init failed, disabled\n", i);
        inputModules[i]->enabled = false;
    }
}
```

Listing 12: init() リトライパターンの実装例

update() エラー時 Data 構造体内のエラーフラグ（hasError 等）を true に設定する。モジュールは次の update() サイクルで再試行し、成功時にモジュール自身がエラーフラグを false にクリアする。

```
void I2cSensorModule::update(SystemData& data) {
    if (_readTimer.getNowTime() >= _config.interval) {
        _readTimer.setTime();
        if (readSensor()) {
            data.i2cSensor.value = _lastValue;
            data.i2cSensor.hasError = false;  // 成功時にクリア
        } else {
            data.i2cSensor.hasError = true;   // 失敗時にセット
```

```
        }
    }
}
```

Listing 13: update() エラーフラグの運用例

4.3.11 init 失敗後の復帰パターン

init() 失敗で enabled = false にしたモジュールは、ロジックフェーズで定期的に init() を再試行し、成功時に enabled = true に復帰させる。

```
// main.cpp
ModuleTimer reinitTimer;

void tryReinitModules() {
    if (reinitTimer.getNowTime() < 5000) return;  // 5秒間隔
    reinitTimer.setTime();

    for (int i = 0; i < INPUT_COUNT; i++) {
        if (!inputModules[i]->enabled) {
            if (inputModules[i]->init()) {
                Serial.printf("[System] Input Module %d: reinit OK\n", i);
                inputModules[i]->enabled = true;
            }
        }
    }
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (!outputModules[i]->enabled) {
            if (outputModules[i]->init()) {
                Serial.printf("[System] Output Module %d: reinit OK\n", i);
                outputModules[i]->enabled = true;
            }
        }
    }
}

void loop() {
    // 1. 入力フェーズ ...
    // 2. ロジックフェーズ
    applyPattern(systemData);
    tryReinitModules();  // 無効モジュールの復帰チェック
    // 3. 出力フェーズ ...
}
```

Listing 14: ロジックフェーズでの init 復帰パターン

4.3.12 ログ出力フォーマット

シリアルモニタへのログ出力は、以下のフォーマットに統一する。

◆ ログフォーマット

[モジュール名] メッセージ

例: [Camera] init failed: timeout

例: [Temp] **センサー読み取りエラー**
例: [System] Input Module 0: reinit OK

デバッグ用のログは#ifdef DEBUG 等で切り替え可能にし、リリースビルドでは不要なログを無効化できるようにする。

4.3.13 Config の実行時変更

Config 構造体は**初期値**として扱う。コンストラクタでコピーされたメンバ変数_config は const ではないため、実行時に変更が必要な場合は**モジュールインスタンス内の_config メンバを直接変更**する。

！ Config 変更のタイミング

Config の変更（セッター呼び出し）は**ロジックフェーズでのみ**行う。BLE 等の外部からの変更要求は SystemData 経由でロジックフェーズに渡し、ロジックフェーズ内でセッターを呼び出す。これにより、update() 実行中に Config が変わるデータ不整合を防ぐ。

```cpp
class TempSensorModule : public IModule {
private:
    TempSensorConfig _config;  // constではない→変更可能
public:
    TempSensorModule(const TempSensorConfig& config)
        : _config(config) {}

    // 実行時にintervalを変更するセッター
    void setInterval(unsigned long interval) {
        _config.interval = interval;
    }

    // update()は変更後の値を使用する
    void update(SystemData& data) override {
        if (_readTimer.getNowTime() >= _config.interval) {
            // ...
        }
    }
};

// ロジックフェーズ等から変更する例
void applyPattern(SystemData& data) {
    if (data.ble.newIntervalReceived) {
        tempModule.setInterval(data.ble.newInterval);
    }
}
```

Listing 15: Config 値の実行時変更例

5 ファイル構成

5.1 標準ディレクトリ構造

本アーキテクチャに準拠するプロジェクトは、以下のディレクトリ構造に従う。

```
ProjectRoot/
+-- include/                    ... プロジェクト固有ヘッダー
|   +-- SystemData.h            ... SystemData構造体（モジュール間共有データ）
|   +-- ProjectConfig.h         ... Configインスタンス定義（ピンアサイン等）
+-- lib/
|   +-- ModuleCore/             ... コアライブラリ（プロジェクト非依存）
|   |   +-- IModule.h           ... IModuleインターフェース
|   |   +-- ModuleTimer.h       ... ModuleTimerクラス
|   +-- {ModuleName}/           ... 各モジュール実装（1モジュール1ディレクトリ）
|       +-- {ModuleName}.h      ... Config/Data構造体 + クラス宣言
|       +-- {ModuleName}.cpp    ... メソッド実装
+-- src/
|   +-- main.cpp                ... エントリーポイント（モジュール登録・ループ制御）
+-- platformio.ini              ... PlatformIO設定（-I include を含む）
```

5.2 ファイル役割一覧

ファイル	種別	説明
`lib/ModuleCore/IModule.h`	コア	IModule インターフェース定義（プロジェクト非依存）
`lib/ModuleCore/ModuleTimer.h`	コア	ModuleTimer クラス定義（プロジェクト非依存）
`lib/{ModuleName}/{ModuleName}.h`	モジュール	Config/Data 構造体の型定義、SystemData 前方宣言、クラス宣言
`lib/{ModuleName}/{ModuleName}.cpp`	モジュール	メソッド実装（SystemData.h をインクルード）
`include/SystemData.h`	プロジェクト固有	各モジュールの Data 構造体を集約した SystemData 構造体
`include/ProjectConfig.h`	プロジェクト固有	モジュール Config インスタンス（ピンアサイン等）
`src/main.cpp`	プロジェクト固有	エントリーポイント（モジュール登録・ループ制御）

5.3 ライブラリ構成方針

- **lib/ModuleCore/** — プロジェクト非依存のコアコンポーネント（IModule、ModuleTimer）。他のプロジェクトにもそのまま流用可能。
- **lib/{ModuleName}/** — 個別のハードウェアモジュール実装。それぞれが `IModule` を継承し、モジュール固有の Config/Data 構造体を定義する。`.h`（宣言 + 前方宣言）と `.cpp`（実装）に分離する。1 モジュール 1 ディレクトリの原則に従う。
- **include/** — プロジェクト固有のヘッダー。`SystemData.h` は使用するモジュールの Data 構造体を集約し、`ProjectConfig.h` は Config インスタンス（ピン番号等の具体的な値）を定義する。

6 技術スタック

カテゴリ	技術	備考
MCU	ESP32-S3 N16R8	デュアルコア Xtensa LX7、16MB Flash、8MB PSRAM
カメラ	OV2640 / OV5640	開発ボード付属
フレームワーク	Arduino	PlatformIO 経由
ビルドツール	PlatformIO	VSCode 統合開発環境
言語	C++	Arduino C++（C++11 以降）
仕様書	LaTeX (uplatex)	DevContainer + LaTeX Workshop
バージョン管理	Git	—

6.1 PlatformIO 設定

```
[env:esp32-s3-cam-n16r8]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

; --- N16 (16MB Flash) ---
board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv

; --- R8 (8MB PSRAM) ---
board_build.arduino.memory_type = qio_opi
build_flags =
    -D BOARD_HAS_PSRAM
    -D CORE_DEBUG_LEVEL=3
    -I include  ; lib/内の.cppからinclude/を参照可能にする
```

Listing 16: platformio.ini

▷ -I include が必要な理由

PlatformIO の標準設定では、lib/内のライブラリはプロジェクトの include/ディレクトリをインクルードパスに含まない。モジュールの.cpp ファイルが SystemData.h を参照するため、build_flags に-I include を追加してパスを通す必要がある。

7 API 仕様

7.1 IModule インターフェース

IModule — モジュールインターフェース

定義ファイル: lib/ModuleCore/IModule.h

全ハードウェアモジュールが実装すべき抽象インターフェース。SystemData の前方宣言のみを使用し、プロジェクト固有のヘッダーに依存しない。

```
// IModule.h — モジュールインターフェース（プロジェクト非依存）
#pragma once

// SystemDataの前方宣言
// 各プロジェクトでSystemData構造体を定義すること
struct SystemData;

class IModule {
public:
    // 仮想デストラクタ
    virtual ~IModule() {}

    // モジュールの初期化
    // 戻り値: true=成功, false=失敗
    virtual bool init() = 0;

    // モジュールの更新（毎ループ呼び出される）
    // 引数: data - プロジェクト共有データへの参照
    virtual void update(SystemData& data) = 0;

    // モジュールの終了処理（リソース解放）
    // デフォルトは空実装。解放が必要なモジュールのみオーバーライド
    virtual void deinit() {}

    // 動的有効/無効化フラグ
    // false の場合、ループ内で update() がスキップされる
    bool enabled = true;
};
```

Listing 17: IModule インターフェース定義

▷ 前方宣言によるプロジェクト非依存性

IModule.h は SystemData を前方宣言のみで参照しているため、SystemData.h を#include する必要がない。

update() の引数は参照型（SystemData&）であるため、前方宣言だけで十分であり、どのプロジェクトにもそのままコピーして流用できる。

！ データアクセスルール

update() 内では、**自モジュールのサブ構造体のみ**にアクセスすること。他モジュールのサブ構造体への読み書きは禁止とする。モジュール間のデータ連携が必要な場合は、必ずロジックフェーズを経由する。

▷ SystemData 名の標準化

IModule.h は前方宣言で SystemData という型名に依存している。この名前は本アーキテクチャの標準命名規則として固定されており、すべてのプロジェクトで SystemData という名前の構造体を include/SystemData.h に定義する。

7.1.1 init()

項目	内容
戻り値	bool — true で初期化成功、false で失敗
呼び出しタイミング	setup() 内で 1 回のみ
責務	ハードウェアの初期化、ピン設定、ModuleTimer の setTime() 呼び出し等
エラー時	false を返す。呼び出し側でモジュールの無効化やログ出力を行う

7.1.2 update(SystemData& data)

項目	内容
引数	SystemData& — プロジェクト共有データへの参照（null にならない）
戻り値	void
呼び出しタイミング	loop() 内で毎ループ呼び出される
責務	センサー読み取り、出力制御、共有データの更新
エラー時	共有データ内のサブ構造体にエラーフラグを設定

7.1.3 deinit()

項目	内容
戻り値	void
呼び出しタイミング	スリープ突入前、モジュール停止時等
責務	ペリフェラルの解放、GPIO 状態のリセット等のクリーンアップ処理
デフォルト実装	空（何もしない）。リソース解放が必要なモジュールのみオーバーライドする

7.1.4 enabled

項目	内容
型	bool — デフォルト値 true
用途	実行時にモジュールを動的に有効化・無効化する
動作	false の場合、ループ内で update() がスキップされる
設定タイミング	init() 失敗時の無効化、BLE 等の外部通信からの切り替え、ロジックフェーズでの制御

7.1.5 使用例

```
// LedModule.h
#pragma once
#include <Arduino.h>
#include "IModule.h"

// モジュール固有の Config 構造体
```

```
struct LedConfig {
    uint8_t ledPin;  // LED出力ピン
};

// モジュール固有のデータ構造体（メンバにデフォルト値を明示）
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

// SystemDataの前方宣言はIModule.h内で行われている

// IModuleを継承
class LedModule : public IModule {
private:
    LedConfig _config;  // Configをメンバとして保持
public:
    // コンストラクタでConfig構造体を受け取る
    LedModule(const LedConfig& config);
    bool init() override;
    void update(SystemData& data) override;
};
```

Listing 18: モジュール実装の例（ヘッダー）

```
// LedModule.cpp
#include "LedModule.h"
#include "SystemData.h"  // SystemDataの完全な定義を取得

LedModule::LedModule(const LedConfig& config) : _config(config) {}

bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);  // Configのピンを使用
    Serial.println("[Led] init OK");
    return true;
}

// data はポインタではなく参照なので、ドット演算子でアクセス
void LedModule::update(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state);
}
```

Listing 19: モジュール実装の例（実装ファイル）

7.2 ModuleTimer クラス

ModuleTimer — ノンブロッキングタイマー

定義ファイル: `lib/ModuleCore/ModuleTimer.h`

`millis()` ベースのシンプルなタイマークラス。`delay()` を使用せずに周期実行・デバウンス・ワンショット・タイムアウト等を実現する。インスタンスを複数作成することで、モジュール内で複数のタイマーを独立に管理できる。IModule と同様にプロジェクト非依存であり、そのまま流用可能。

```cpp
// ModuleTimer.h — プロジェクト非依存のコアヘッダー
#pragma once
#include <Arduino.h>

class ModuleTimer {
private:
    unsigned long _startTime;

public:
    // コンストラクタ
    ModuleTimer() : _startTime(0) {}

    // 基準時刻をセット
    // 引数なし: getNowTime()が0から開始
    // 引数あり: getNowTime()が指定値から開始
    void setTime(unsigned long offsetMs = 0) {
        _startTime = millis() - offsetMs;
    }

    // 基準時刻からの経過時間(ms)を取得
    unsigned long getNowTime() const {
        return millis() - _startTime;
    }
};
```

Listing 20: ModuleTimer クラス定義

7.2.1 setTime(unsigned long offsetMs = 0)

項目	内容
引数	`offsetMs`（デフォルト: 0） — 初期オフセット値 (ms)
戻り値	`void`
動作	内部の基準時刻を `millis() - offsetMs` に設定する。`offsetMs = 0` の場合、直後の `getNowTime()` は 0 を返す。`offsetMs = 100` の場合、直後の `getNowTime()` は 100 を返す
用途	タイマーの開始・リセット。`init()` 内および周期リセット時に使用

7.2.2 getNowTime() const

項目	内容
引数	なし
戻り値	`unsigned long` — 最後の `setTime()` からの経過時間 (ms)
動作	`millis() - _startTime` を返す
用途	経過時間の判定。条件分岐と組み合わせて周期実行・デバウンス等を実現

7.2.3 使用パターン

```cpp
ModuleTimer readTimer;

bool init() override {
```

```
    readTimer.setTime();
    return true;
}

void update(SystemData& data) override {
    if (readTimer.getNowTime() >= 1000) {
        readTimer.setTime();  // リセット
        data.temp.temperature = readSensor();
    }
}
```

Listing 21: 周期実行パターン

```
ModuleTimer debounce;

void update(SystemData& data) override {
    bool rawInput = digitalRead(BUTTON_PIN);

    if (rawInput != lastRawInput) {
        debounce.setTime();  // 入力変化でリセット
        lastRawInput = rawInput;
    }

    if (debounce.getNowTime() >= 50) {
        data.button.pressed = lastRawInput;  // 50ms安定で確定
    }
}
```

Listing 22: デバウンスパターン

7.3 ProjectConfig ファイルと SystemData

ProjectConfig — Config インスタンスの集約

定義ファイル: `include/ProjectConfig.h`（プロジェクト固有）
各モジュールの Config インスタンス（ピンアサイン等）を集約した、プロジェクト固有の設定ファイル。`main.cpp` のみがインクルードする。

SystemData — モジュール間共有データ構造体

定義ファイル: `include/SystemData.h`（プロジェクト固有）
各モジュールの Data 構造体を集約した共有データ構造体。`IModule` の `update()` メソッドの引数型として使用される。モジュールの `.cpp` ファイルからインクルードされる。

7.3.1 設計ルール

1. 各モジュールは、自身の `Config` 構造体（型定義）と `Data` 構造体（型定義）をモジュールのヘッダーファイル内に定義する
2. `ProjectConfig.h` には、Config の**インスタンス**（実際のピン番号等の値）のみを定義する
3. `SystemData` 構造体は `SystemData.h` に分離し、プロジェクトで使用するモジュールの Data 構造体をメンバとして持つ

4. モジュールの.cpp は SystemData.h をインクルードする（ProjectConfig.h はインクルードしない）
5. モジュールは update() 内で、**自身の Data 構造体のみ**にアクセスする（他モジュールの Data 構造体への読み書きは禁止）

```
// SystemData.h — モジュール間共有データ構造体
#pragma once
#include "LedModule.h"
#include "TempSensorModule.h"
#include "CameraModule.h"

struct SystemData {
    LedData led;
    TempData temp;
    CameraData camera;
};
```

Listing 23: SystemData.h 構成例

```
// ProjectConfig.h — Configインスタンスのみ
#pragma once
#include "SystemData.h"

// ===== モジュールConfig（ピンアサイン・設定値） =====
const LedConfig LED_CONFIG = { .ledPin = 13 };
const TempSensorConfig TEMP_CONFIG = { .tempPin = 36 };
const CameraConfig CAMERA_CONFIG = { .sdaPin = 21, .sclPin = 22 };
```

Listing 24: ProjectConfig.h 構成例

▷ ファイル分離の理由

SystemData.h を ProjectConfig.h から分離することで、モジュールの.cpp ファイルは ProjectConfig.h を知る必要がなくなる。

SystemData.h — 各モジュールの Data 構造体を集約。モジュールの.cpp がインクルード。

ProjectConfig.h — Config インスタンス（ピン番号等の具体的な値）のみ。main.cpp だけがインクルードすればよい。回路設計担当者はこのファイルの Config 定義だけを参照すれば回路設計が可能。

7.4 モジュール Config 構造体

モジュール Config — ハードウェア設定構造体

各モジュールが必要とするピンアサインや設定をまとめた構造体。各モジュールのヘッダーファイル内で型を定義し、ProjectConfig.h 内で具体的な値を持つインスタンスを定義する。

7.4.1 命名規則

要素	命名規則	例
Config 構造体の型名	{Module}Config	LedConfig, CameraConfig
Config インスタンス名	{MODULE}_CONFIG(大文字)	LED_CONFIG, CAMERA_CONFIG
Data 構造体の型名	{Module}Data	LedData, CameraData

7.4.2 設計ルール

1. Config 構造体の**型定義**は、各モジュールのヘッダーファイル内に記述する
2. Config 構造体の**インスタンス**（実際のピン番号等）は `ProjectConfig.h` に集約する
3. モジュールのコンストラクタは `const {Module}Config&`（const 参照）で Config を受け取る
4. 受け取った Config は、メンバ変数 `_config` としてコピー保持する（Config は初期値として扱う）
5. 実行時に設定値の変更が必要な場合は、`_config` メンバを直接変更するか、セッターメソッドを用意する
6. ピン番号はハードコードせず、必ず `_config` 経由でアクセスする
7. 通信バスを使用するモジュールは、バスポインタを Config 構造体に含めず、コンストラクタの引数で受け取る（§通信バスパターン参照）

```
// TempSensorModule.h
#pragma once
#include "IModule.h"
#include "ModuleTimer.h"

// --- Config構造体（ピンアサイン・設定） ---
struct TempSensorConfig {
    uint8_t tempPin;          // アナログ入力ピン
    unsigned long interval;   // 読み取り間隔(ms)
};

// --- Data構造体（メンバにデフォルト値を明示） ---
struct TempData {
    float temperature = -999.0f;
    bool isValid = false;
};

class TempSensorModule : public IModule {
private:
    TempSensorConfig _config;
    ModuleTimer _readTimer;
public:
    // コンストラクタ: Config構造体をconst参照で受け取る
    TempSensorModule(const TempSensorConfig& config)
        : _config(config) {}

    bool init() override {
        pinMode(_config.tempPin, INPUT);
        _readTimer.setTime();
        return true;
    }

    void update(SystemData& data) override {
        if (_readTimer.getNowTime() >= _config.interval) {
            _readTimer.setTime();
            data.temp.temperature = analogRead(_config.tempPin);
            data.temp.isValid = true;
        }
    }
};
```

Listing 25: Config 構造体の定義と使用例

```cpp
// SpiSensorModule.h — Config構造体にはバスポインタを含めない
struct SpiSensorConfig {
    uint8_t csPin;            // チップセレクトピン
    uint32_t clockSpeed;      // SPIクロック速度(Hz)
    unsigned long interval;   // 読み取り間隔(ms)
};

class SpiSensorModule : public IModule {
private:
    SpiSensorConfig _config;
    SPIClass* _spi;  // バスポインタはメンバ変数として保持
public:
    // Config + バスポインタをコンストラクタで受け取る
    SpiSensorModule(const SpiSensorConfig& config, SPIClass* spi)
        : _config(config), _spi(spi) {}
};

// ProjectConfig.h — Configは純粋な設定値のみ
const SpiSensorConfig SPI_SENSOR_CONFIG = {
    .csPin = 5, .clockSpeed = 1000000, .interval = 100
};

// main.cpp — バスの紐付けはモジュール生成時に行う
SPIClass spi(HSPI);
SpiSensorModule spiModule(SPI_SENSOR_CONFIG, &spi);
```

Listing 26: 通信バスを使用するモジュールの Config 構造体とコンストラクタ例

8 実行モデル詳細

8.1 ライフサイクル

本アーキテクチャのモジュールは、以下のライフサイクルに従う：

1. **インスタンス生成**（グローバルスコープ）
   モジュールのコンストラクタが呼ばれる。`ProjectConfig.h` で定義した Config 定数を渡し、メンバ変数_config にコピーされる。ModuleTimer の_startTime = 0 で初期化される。この時点では `millis()` は信頼できないため、タイマーの実質的な初期化は行わない。
2. **バス初期化**（`setup()` 内の最初）
   SPI・I2C 等の通信バスを使用する場合、`SPI.begin()` や `Wire.begin()` 等のバス初期化を全モジュールの `init()` より前に実行する。バスインスタンスは `main.cpp` のグローバルスコープで生成し、Config 構造体のポインタ経由でモジュールに渡す。
3. **init() 呼び出し**（`setup()` 内）
   ハードウェアの初期化、ピンモード設定、ModuleTimer の `setTime()` 呼び出しを行う。失敗時は `false` を返し、呼び出し側でエラーハンドリングを行う。
4. **update() ループ**（`loop()` 内）
   毎ループ呼び出される。`SystemData` の参照を受け取り、ModuleTimer の `getNowTime()` で経過時間を確認し、必要なタイミングでのみ実処理を実行する。

5. **deinit() 呼び出し**（必要時）
   スリープ突入前やモジュール停止時に呼び出す。ペリフェラルの解放、GPIO 状態のリセット等のクリーンアップ処理を行う。デフォルトは空実装であり、リソース解放が必要なモジュールのみオーバーライドする。

8.2 ループ実行手順（3 フェーズモデル）

`loop()` 内の処理は、以下の 3 フェーズに分けて実行する。この順序を守ることで、モジュール間の暗黙的な依存関係を排除し、各モジュールの独立性を保つ。

1. **入力フェーズ** — センサー・スイッチ等の入力系モジュールの `update()` を実行。ハードウェアから値を読み取り、自身の `SystemData` サブ構造体に書き込む。
2. **ロジックフェーズ** — 入力系の `SystemData` を読み取り、出力系の `SystemData` を書き換える制御ロジックを実行する。
3. **出力フェーズ** — LED・モーター等の出力系モジュールの `update()` を実行。自身の `SystemData` サブ構造体の値に従ってハードウェアを制御する。

▷ 3 フェーズの原則

各モジュールは「それ単体で動作する道具」である。
入力モジュールは「値を読んで SystemData に書く」だけ、出力モジュールは「SystemData の値に従って動作する」だけ。
モジュールの update() 内で他モジュールの SystemData を参照してはならない。
「何をどう制御するか」の判断はロジックフェーズの責務である。

8.2.1 ロジックフェーズの実装

ロジックフェーズの具体的な実装方法は開発者に委ねるが、標準的には関数として実装する。

```cpp
// main.cpp
void applyPattern(SystemData& data) {
    // 入力SystemDataを読み、出力SystemDataを書き換える
    // 例：スイッチが押されたらLEDを点灯
    data.led.state = data.sw.isPressed;

    // 例：温度が30度を超えたらモーターを回す
    if (data.temp.isValid && data.temp.temperature > 30.0) {
        data.motor.speed = 255;
    } else {
        data.motor.speed = 0;
    }
}

void loop() {
    // 1. 入力フェーズ
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {
            inputModules[i]->update(systemData);
        }
    }
    // 2. ロジックフェーズ
    applyPattern(systemData);
    // 3. 出力フェーズ
```

```
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (outputModules[i]->enabled) {
            outputModules[i]->update(systemData);
        }
    }
}
```

Listing 27: ロジックフェーズの標準的な実装

8.2.2 入出力分離ルール

1 つのデバイスが入力と出力の両方の機能を持つ場合、それぞれを**別モジュールとして実装**する。

デバイス	入力モジュール	出力モジュール
エンコーダ付きモーター	EncoderModule（回転数取得）	MotorModule（モーター制御）
タッチ＋振動デバイス	TouchModule（タッチ検出）	HapticModule（振動出力）

これにより、各モジュールの責務が「入力のみ」または「出力のみ」に限定され、モジュールの独立性が保たれる。

8.2.3 アンチパターン

出力モジュール内で入力系の `SystemData` を読み取ってロジックを書く行為は**禁止**とする。

```
// NG: LEDモジュール内で温度を判断している
void LedModule::update(SystemData& data) {
    // 他モジュールのSystemDataを読んではならない
    if (data.temp.temperature > 30.0) {
        data.led.state = true;
    }
    digitalWrite(_config.ledPin, data.led.state);
}
```

Listing 28: アンチパターン：出力モジュール内でロジックを持つ（禁止）

```
// OK: LEDモジュールは自身のデータだけを見る
void LedModule::update(SystemData& data) {
    // 自身のサブ構造体の値に従って動作するだけ
    digitalWrite(_config.ledPin, data.led.state);
}

// ロジックは別の場所（applyPattern等）で行う
void applyPattern(SystemData& data) {
    data.led.state = (data.temp.temperature > 30.0);
}
```

Listing 29: 正しい実装：出力モジュールは自身の SystemData のみ参照

8.3 タイミング制御

`delay()` は一切使用しない。すべてのタイミング制御は ModuleTimer で行う。

パターン	方法	コード例
周期実行	getNowTime() >= interval で判定し、setTime() でリセット	1 秒ごとのセンサー読み取り
デバウンス	入力変化時に setTime() でリセット、安定後に確定	ボタンのチャタリング除去
ワンショット	getNowTime() で経過時間を確認、フラグで制御	起動後 N 秒で LED 消灯
タイムアウト	getNowTime() >= timeout で時間超過を検出	WiFi 接続のタイムアウト

8.4 モジュール間データフロー

3 フェーズモデルにおける SystemData を介したデータフローの例：

```
// --- 入力フェーズ ---
// 温度センサーモジュール：ハードウェアから値を読み、自身のデータに書く
void TempSensorModule::update(SystemData& data) {
    if (_readTimer.getNowTime() >= _config.interval) {
        _readTimer.setTime();
        data.temp.temperature = analogRead(_config.tempPin);
        data.temp.isValid = true;
    }
}

// --- ロジックフェーズ（main.cpp内の関数等） ---
void applyPattern(SystemData& data) {
    // 温度に応じてLEDを制御する判断をここで行う
    if (data.temp.isValid && data.temp.temperature > 30.0) {
        data.led.state = true;
    } else {
        data.led.state = false;
    }
}

// --- 出力フェーズ ---
// LEDモジュール：自身のデータに従って出力するだけ
void LedModule::update(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state);
}
```

Listing 30: 3 フェーズモデルによるデータフロー

8.5 割り込み実行パターン

リアルタイム性が求められるプロジェクトや、カメラ等の重い処理を含むプロジェクトでは、3 フェーズループをタイマー割り込みで周期実行し、メインループは重い処理に専念する構成をとる。

```
// main.cpp
hw_timer_t* moduleTimer = NULL;
volatile bool updateFlag = false;

// タイマー割り込みハンドラ
void IRAM_ATTR onModuleTimer() {
    updateFlag = true;
}

```

```
void setup() {
    Serial.begin(115200);

    // モジュール初期化（省略）

    // タイマー割り込み設定（10ms周期）
    moduleTimer = timerBegin(0, 80, true);  // 80分周 = 1MHz
    timerAttachInterrupt(moduleTimer, &onModuleTimer, true);
    timerAlarmWrite(moduleTimer, 10000, true);  // 10ms
    timerAlarmEnable(moduleTimer);
}

void loop() {
    // メインループ：重い処理（カメラ取得等）を実行
    cameraModule.capture();  // 時間のかかる処理

    // 割り込みフラグで軽量モジュール更新
    if (updateFlag) {
        updateFlag = false;
        // 1. 入力フェーズ
        for (int i = 0; i < INPUT_COUNT; i++) {
            if (inputModules[i]->enabled) {
                inputModules[i]->update(systemData);
            }
        }
        // 2. ロジックフェーズ
        applyPattern(systemData);
        // 3. 出力フェーズ
        for (int i = 0; i < OUTPUT_COUNT; i++) {
            if (outputModules[i]->enabled) {
                outputModules[i]->update(systemData);
            }
        }
    }
}
```

Listing 31: タイマー割り込みによる 3 フェーズ実行

▷ 割り込み実行パターンの使い分け

シンプルなプロジェクトでは通常の `loop()` 内での 3 フェーズ実行で十分である。
割り込み実行パターンは、重い処理（カメラ・画像処理・FFT 等）と軽量な周期処理（センサー読み取り・LED 制御等）を共存させる必要がある場合に使用する。

8.6 外部データアクセスパターン（スレッドセーフティ）

3 フェーズループの外（割り込み・別タスク・BLE コールバック等）から `SystemData` に直接アクセスしてはならない。外部からのデータは、**モジュール内部バッファ** ＋ `volatile` **フラグ方式**で受け渡す。割り込みでもマルチタスクでも同じルールを適用する。

```
class BleModule : public IModule {
private:
    BleConfig _config;
```

```
    // 内部バッファ（BLEコールバックが書き込む）
    uint8_t _rxBuffer[128];
    uint8_t _rxLength = 0;
    volatile bool _dataReceived = false;  // volatileフラグ

public:
    BleModule(const BleConfig& config) : _config(config) {}

    // BLEコールバックから呼ばれる（別タスクで実行される）
    void onReceive(uint8_t* data, uint8_t length) {
        memcpy(_rxBuffer, data, length);  // バッファに書き込み
        _rxLength = length;
        _dataReceived = true;             // フラグを立てる
    }

    // update()は3フェーズループ内で呼ばれる
    void update(SystemData& sysData) override {
        if (_dataReceived) {
            // フラグが立っていたらバッファからSystemDataへコピー
            memcpy(sysData.ble.rxData, _rxBuffer, _rxLength);
            sysData.ble.rxLength = _rxLength;
            sysData.ble.newData = true;
            _dataReceived = false;        // フラグをクリア
        }
    }
    // ...
};
```

Listing 32: 外部データアクセスパターン（volatile フラグ方式）

▷ volatile フラグ方式を使う理由

volatile は「この変数は外部から変更される可能性がある」とコンパイラに伝えるキーワードである。これがないと、コンパイラの最適化によりフラグの変更が検出されない場合がある。

bool 型フラグ 1 個の読み書きは原子的であるため、「バッファへの書き込み完了後にフラグを立てる」→「フラグ確認後にバッファを読む」の順序で安全にデータを受け渡しできる。mutex は不要である。

！ SystemData への直接アクセス禁止

以下のコードは**禁止**である。BLE コールバックや割り込みハンドラから SystemData を直接書き換えると、3 フェーズループ実行中にデータが変わり不整合が発生する。

```
// NG: コールバックからSystemDataに直接書き込み
void onBleReceive(SystemData& data, uint8_t* payload) {
    data.ble.command = payload[0];  // 禁止
}
```

8.7 内部バッファパターン

カメラ等の重い処理を持つモジュールでは、メインループで取得したデータをクラス内バッファに保存し、update() 呼び出し時に SystemData へコピーする。これにより、update() 自体は軽量な処理に留まる。

```
// CameraModule.h
struct CameraData {
    uint8_t* frame = nullptr;     // フレームデータポインタ
    size_t frameSize = 0;         // フレームサイズ
    bool newFrame = false;        // 新規フレームフラグ
    bool hasError = false;
};

class CameraModule : public IModule {
private:
    CameraConfig _config;
    // 内部バッファ
    uint8_t* _buffer = nullptr;
    size_t _bufferSize = 0;
    volatile bool _bufferReady = false;

public:
    CameraModule(const CameraConfig& config)
        : _config(config) {}

    bool init() override {
        // カメラハードウェア初期化
        // ...
        return true;
    }

    // メインループから直接呼ぶ（重い処理）
    void capture() {
        // カメラからフレーム取得（時間がかかる）
        camera_fb_t* fb = esp_camera_fb_get();
        if (fb) {
            _buffer = fb->buf;
            _bufferSize = fb->len;
            _bufferReady = true;
        }
    }

    // update()は軽量：バッファからSystemDataへコピーするだけ
    void update(SystemData& data) override {
        if (_bufferReady) {
            data.camera.frame = _buffer;
            data.camera.frameSize = _bufferSize;
            data.camera.newFrame = true;
            _bufferReady = false;
        }
    }
};
```

Listing 33: 内部バッファパターンの実装例

◆ 内部バッファパターンの流れ

1. loop() のメイン処理で capture() を呼び、重い処理を実行
2. 処理完了時にクラス内バッファにデータを保存し、_bufferReady = true にセット
3. タイマー割り込みの 3 フェーズループで update() が呼ばれる
4. update() はバッファから SystemData へコピーするだけ（軽量）

8.8 モジュール実装テンプレート

新規モジュールを追加する際の標準テンプレート：

```
// ExampleModule.h
#pragma once
#include "IModule.h"
#include "ModuleTimer.h"

// --- Config構造体（ピンアサイン・設定） ---
struct ExampleConfig {
    uint8_t inputPin;          // 入力ピン
    unsigned long interval;    // 更新間隔(ms)
};

// --- Data構造体（メンバにデフォルト値を明示） ---
struct ExampleData {
    int value = 0;
    bool hasError = false;
};

// IModule を継承
class ExampleModule : public IModule {
private:
    ExampleConfig _config;      // Configをメンバとして保持
    ModuleTimer _updateTimer;   // タイマー

public:
    // コンストラクタ: Config構造体をconst参照で受け取る
    ExampleModule(const ExampleConfig& config)
        : _config(config) {}

    bool init() override {
        pinMode(_config.inputPin, INPUT);
        _updateTimer.setTime();  // タイマー初期化
        return true;
    }

    void update(SystemData& data) override {
        if (_updateTimer.getNowTime() >= _config.interval) {
            _updateTimer.setTime();
            // data.example.value = ...;  // 更新処理
        }
    }
};
```

Listing 34: モジュール実装テンプレート

9 既知の制約

本アーキテクチャは小〜中規模（モジュール 10 個程度まで）のプロジェクトに適した設計である。以下に、設計上認識している制約を示す。各制約に対する対応方針は、該当するセクションに正式な設計ルールとして記載している。

9.1 SystemData の肥大化

モジュールが増えるたびに `SystemData` にサブ構造体が追加され、構造体が肥大化する。全モジュールが `SystemData&`を受け取るため、自身に無関係なデータにもアクセス可能になる。

▷ 肥大化の許容範囲

モジュール数が 15〜20 を超えると構造体の見通しが悪化するが、本アーキテクチャの対象規模（10 個程度）では問題にならない。モジュール 1 つの追加で `ProjectConfig.h` の変更が必要となり全モジュールの再コンパイルが発生する点は、小〜中規模プロジェクトでは許容範囲とする。
対応方針は「システムアーキテクチャ」§データアクセスルールを参照。

9.2 モジュール間の依存関係

現在のアーキテクチャでは、モジュール間の初期化順序や実行順序の依存関係を明示的に表現する仕組みがない。
対応方針は「システムアーキテクチャ」§モジュール統合の原則を参照。

9.3 リアルタイム性の限界

3 フェーズモデルは処理の流れが明快であるが、1 ループ周期分の応答遅延が必ず発生する。
対応方針は「実行モデル詳細」§割り込み実行パターンを参照。

9.4 シングルスレッド制約

本アーキテクチャは協調的なシングルスレッドスケジューリングに基づいており、カメラ画像処理等の重い処理がモジュールに含まれると全体が遅延する。また、割り込みや別タスクからの `SystemData` アクセスにはデータ競合のリスクがある。
対応方針は「実行モデル詳細」§内部バッファパターンおよび§外部データアクセスパターンを参照。

9.5 ロジックフェーズのスケーラビリティ

`applyPattern()` 等の関数でロジックを記述する方式は、モジュール数の増加に伴い複雑化する可能性がある。ただし、すべてのモジュール間データ連携は `SystemData` を介して行うため、モジュール同士が直接競合することはなく、競合が発生した場合はロジック関数内で優先度を明示的にコード化すればよい。

9.6 動的なモジュール管理

モジュール配列は `setup()` 時に静的に確定され、実行中にモジュールを追加・削除する仕組みがない。
対応方針は「システムアーキテクチャ」§ enabled フラグによる動的管理を参照。

9.7 エラー復帰パターン

エラーの検出手段は提供されているが、エラーからの復帰に関するパターンが未定義であった。
対応方針は「システムアーキテクチャ」§エラーハンドリングパターンを参照。

9.8 Config の実行時変更

Config 構造体はコンストラクタでコピーされ、以降の変更手段が限られていた。
対応方針は「システムアーキテクチャ」§ Config の実行時変更を参照。

9.9 テスタビリティ

ハードウェア依存が強く、PC 上でのユニットテストが困難である。本アーキテクチャは Arduino プラットフォームでのプロジェクトが大前提であり、テストはハードウェア実機上での動作確認（シリアルモニタ出力等）を基本とする。

9.10 メモリ管理

大容量バッファ（カメラフレーム等）の確保先について、本アーキテクチャでは特定の方式を強制しないが、ESP32-S3 等の PSRAM 搭載ボードでは大容量バッファに PSRAM の使用を推奨する。

▷ 本セクションの位置づけ

ここに挙げた制約の多くは、アーキテクチャの設計上の欠陥ではなく、**簡潔さとのトレードオフ**として認識したものである。各制約に対する対応方針は、アーキテクチャ・実行モデル・API 仕様の各セクションに正式な設計ルールとして記載している。